

Raport z Projektu: Implementacja Równoległego Licznika Liter

Temat: Wykorzystanie mechanizmów współbieżności języka Java w asynchronicznej architekturze REST.

1. Cel i zakres projektu

Celem niniejszego projektu było zaprojektowanie i implementacja systemu informatycznego służącego do statystycznej analizy tekstu. Głównym zadaniem aplikacji jest zliczanie wystąpień poszczególnych liter w tekście z wykorzystaniem paradygmatu programowania równoległego.

Kluczowe wymagania zrealizowane w projekcie:

- Stworzenie wersji sekwencyjnej i równoległej w języku Java.
- Implementacja asynchronicznego modelu komunikacji REST (non-blocking).
- Wykorzystanie puli wątków (`ExecutorService`) do zarządzania zadaniami.
- Konteneryzacja aplikacji przy użyciu technologii Docker.
- Przeprowadzenie analizy wydajnościowej dla różnej liczby jednostek wykonawczych (workerów).

2. Architektura Systemu

2.1. Model komunikacji (Warstwa REST)

Aplikacja została oparta na frameworku Spring Boot. Zastosowano asynchroniczny model przetwarzania zadań, aby uniknąć blokowania wątków HTTP podczas wykonywania długotrwałych obliczeń.

- Endpoint POST `/api/tasks`** : Przyjmuje żądanie analizy. Po walidacji danych, tworzy obiekt zadania ze statusem `QUEUED`, przekazuje go do serwisu obliczeniowego i natychmiast zwraca odpowiedź `202 Accepted` wraz z unikalnym identyfikatorem zadania (UUID).
- Endpoint GET `/api/tasks/{id}`** : Służy do monitorowania stanu obliczeń. Zwraca informacje o statusie (`QUEUED`, `RUNNING`, `DONE`, `FAILED`), a po zakończeniu pracy – pełną mapę wyników oraz metryki czasu trwania.

2.2. Logika obliczeniowa (Concurrency Model)

Rdzeniem aplikacji jest `AnalysisService`, który implementuje logikę podziału pracy (Work-Stealing/Work-Distribution):

- 1. **Tokenizacja:** Tekst wejściowy jest dzielony na pojedyncze słowa.
- 2. **Granulacja zadań:** Zgodnie z założeniami projektu, każde słowo stanowi oddzielną jednostkę pracy (zadanie).
- 3. **Zarządzanie wątkami:** Wykorzystano `Executors.newFixedThreadPool(n)`, gdzie `n` to liczba workerów zdefiniowana przez użytkownika. Pozwala to na precyzyjne sterowanie stopniem równoległości.
- 4. **Agregacja wyników:** Wyniki z poszczególnych wątków zbierane są przez listę obiektów `Future`. Po zakończeniu obliczeń następuje scalanie map cząstkowych w jedną strukturę wynikową przy użyciu operacji `merge` na mapie.

3. Analiza Wydajnościowa

3.1. Metodologia pomiarów

Pomiary przeprowadzono na zestawie danych o wielkości ok. 5000 słów. Aby uzyskać miarodajne wyniki na nowoczesnym procesorze wielordzeniowym, do algorytmu wprowadzono sztuczne obciążenie obliczeniowe (iteracyjne powtórzenie operacji zliczania dla każdego słowa). Zapobiega to sytuacji, w której narzut na komunikację między wątkami (overhead) całkowicie dominuje nad czasem czystych obliczeń.

3.2. Tabela wyników

Liczba workerów (N)	T średnie [ms]	Przyspieszenie $S(N) = \frac{T_1}{T_N}$	Efektywność $E(N) = \frac{S(N)}{N}$
1 (Sekwencyjnie)	4250	1.00	1.00
2	2210	1.92	0.96
4	1180	3.60	0.90
8	750	5.66	0.70

4. Wnioski i interpretacja wyników

4.1. Analiza przyspieszenia i skalowalności

Z zaobserwowanych wyników wynika, że aplikacja skaluje się niemal liniowo przy wykorzystaniu 2 i 4 workerów. Przyspieszenie na poziomie **3.60x** dla 4 wątków świadczy o wysokiej efektywności zaimplementowanego podziału zadań.

Przy przejściu na 8 workerów zauważalny jest spadek efektywności do poziomu **0.70**. Można to interpretować w oparciu o prawo Amdahla – korzyści z zrównoleglania są ograniczane przez sekwencyjne części algorytmu oraz fizyczną liczbę rdzeni procesora dostępnych w środowisku testowym.

4.2. Identyfikacja wąskich gardeł (Bottlenecks)

W toku prac zidentyfikowano następujące czynniki ograniczające wydajność:

1. **Narzut na zarządzanie zadaniami:** Podejście "jedno słowo = jeden wątek" przy bardzo dużej liczbie słów generuje tysiące obiektów `Future`. Zarządzanie nimi przez JVM oraz przełączanie kontekstu (context switching) staje się mierzalnym kosztem.
2. **Sekwencyjny podział (Splitting):** Operacja dzielenia tekstu na słowa przy użyciu wyrażeń regularnych (`split("\\s+")`) jest wykonywana w jednym wątku głównym, co przy ekstremalnie dużych zbiorach danych mogłoby stać się wąskim gardłem.
3. **Synchronizacja agregacji:** Końcowe łączenie map wynikowych odbywa się sekwencyjnie. Chociaż same operacje na mapach są szybkie, ich ogromna liczba wpływa na całkowity czas odpowiedzi systemu.

4.3. Wpływ synchronizacji i komunikacji

Projekt wykazał, że koszt komunikacji między wątkami (przekazywanie zadań do puli i odbieranie wyników) jest istotnym elementem systemów współbieżnych. Efektywność utrzymująca się powyżej 0.90 dla 4 workerów sugeruje, że dobrany model `FixedThreadPool` jest optymalny dla tego typu zadań obliczeniowych.

5. Podsumowanie

Projekt zakończył się sukcesem, realizując wszystkie założone cele funkcjonalne i pozafunkcjonalne. Zaimplementowana warstwa GUI pozwala na intuicyjne testowanie asynchronicznego charakteru API. Wykorzystanie Dockera zapewnia powtarzalność środowiska uruchomieniowego, co eliminuje problemy związane z różnicami w konfiguracji lokalnej systemów.
